End-of-Studies Project · Statement of Work

Cloud Data Platform
Architecture

Multi-Cloud · Kubernetes-Native · Lakehouse

Platform	AWS EKS + On-Prem Ready
Storage	MinIO (S3-Compatible)
Compute	Apache Spark + dbt
Catalog	Hive Metastore + Iceberg
BI Layer	Dremio + Power BI
Exposure	Cloudflare Zero Trust
IaC	Terraform + Ansible
CI/CD	GitHub Actions

Released by : Mohammed Dahiry

Academic Year 2025–2026 · MVP Scope

Contents

1 Project Overview & Motivation

1.1 Context and Background

Modern enterprises generate vast quantities of data from heterogeneous sources—relational databases, flat files, streaming event queues, and SaaS applications. Building a cohesive, production-grade data infrastructure that can ingest, store, transform, and expose this data at scale is one of the most pressing engineering challenges of the current decade.

This End-of-Studies Project (PFE) designs, implements, and validates a **Cloud Data Platform** that addresses this challenge through a modern, Kubernetes-native, multi-cloud approach. The platform is grounded in the **Lakehouse** paradigm—combining the cost efficiency and flexibility of a Data Lake with the transactional reliability and query performance of a Data Warehouse.

What is a Modern Data Platform?

A **Modern Data Platform** (MDP) unifies data ingestion, storage, transformation, serving, and observability into a coherent architecture. Key characteristics include:

- **Open standards**: Apache Iceberg table format, Parquet/ORC files, open catalogs.
- **Decoupled storage and compute**: Scale each independently.
- **Cloud-agnostic**: Avoid vendor lock-in through S3-compatible APIs and container portability.
- **Declarative IaC**: Infrastructure as Code with Terraform; configuration with Ansible.
- **Observable**: Metrics, logs, and traces for every layer.

1.2 Problem Statement

Organizations frequently face:

1. **Data Silos**: Data locked in separate databases (PostgreSQL, SQL Server, IBM

Informix) with no unified access layer.

2. **Fragile Pipelines**: Ad-hoc ETL scripts lacking orchestration, lineage, and schema evolution support.
3. **Vendor Lock-in**: Heavy dependence on a single cloud provider for storage and compute.
4. **Lack of Observability**: No centralized logging, monitoring, or alerting across the data stack.
5. **Cost Sprawl**: Cloud resources running 24/7 regardless of actual usage.

This project proposes a holistic solution to all five problems within a time-boxed MVP scope.

1.3 Project Objectives

Primary Objectives

1. Deploy a **Kubernetes-native Data Lakehouse** on AWS EKS using Operator patterns.
2. Implement a **Medallion Architecture** (Bronze / Silver / Gold) backed by Apache Iceberg on MinIO.
3. Build **ingestion pipelines** (NiFi + Kafka + Spark) for CDC, batch, and file-based sources.
4. Expose data through **Dremio** (SQL engine) to **Power BI** for business analytics.
5. Establish full **observability** via Prometheus, Grafana, and OpenObserve.
6. Manage all infrastructure with **Terraform** and **Ansible**, version-controlled in GitHub.
7. Implement **cost controls** via scheduled EKS node scaling (Lambda-based).

2 Architecture Design

2.1 High-Level Architecture

The platform is organized into seven distinct functional layers, each independently scalable and loosely coupled via well-defined interfaces:

Table 2.1: Architecture Layer Summary

Layer	Responsibility	Key Technologies
Data Sources	Origin OLTP databases & flat files	PostgreSQL, SQL Server, IBM Informix, CSV/XLSX
Ingestion	CDC, batch & file ingestion	Apache NiFi, Apache Kafka, Spark Structured Streaming
Storage	S3-compatible Lakehouse storage	MinIO, Apache Iceberg, Hive Metastore
Compute	Transformation & orchestration	Apache Spark, dbt, Apache Airflow, JupyterHub
Virtualization	SQL query federation	Dremio Nautilus
Presentation	BI & self-service analytics	Power BI
Observability	Metrics, logs, dashboards	Prometheus, Grafana, OpenObserve, Fluent Bit

2.2 Kubernetes Namespace Strategy

All workloads run on **AWS EKS** and are organized into logical Kubernetes namespaces for isolation, RBAC, and resource quotas:

Namespace Layout

Namespace	Workloads	Owner
platform-ingestion	NiFi, Kafka, Kafka Connect	Data Engineering
platform-storage	MinIO Operator, MinIO Tenant	Infra Team
platform-metastore	Hive Metastore, PostgreSQL (HMS)	Data Engineering
platform-compute	Spark Operator, dbt jobs	Data Engineering
platform-orchestr	Airflow, JupyterHub	Data Engineering
platform-serving	Dremio, HMS	Analytics
platform-monitoring	Prometheus, Grafana, OTel	SRE
platform-logging	Fluent Bit, OpenObserve	SRE
platform-security	Cloudflare Operator, cert-manager	Infra Team

2.3 Data Flow and Medallion Architecture

The storage layer is organized following the **Medallion Architecture**, a staged data quality approach:

Bronze Raw ingested data in its original fidelity. Written as Iceberg tables. No transformations applied. Immutable once written.

Silver Cleansed, deduplicated, and joined data. Schema is enforced. Suitable for data analysts and data scientists.

Gold Aggregated, business-metric-ready datasets optimized for BI tools. Power BI reads from this layer via Dremio.

The three tiers map directly to MinIO bucket prefixes:

s3a://lakehouse/bronze/ → s3a://lakehouse/silver/ → s3a://lakehouse/gold/

3 Infrastructure Layer – Kubernetes Operators

3.1 Operator Pattern Overview

The **Kubernetes Operator Pattern** extends the Kubernetes API with custom controllers that encode operational knowledge for specific software. Instead of managing pods manually, operators allow declaring the desired state of complex stateful systems as **Custom Resource Definitions (CRDs)**.

> **Core principle:** Every stateful component in this platform (PostgreSQL, MinIO, Spark, Kafka) is managed by a dedicated Kubernetes Operator. No manual pod configuration. No hand-crafted StatefulSets. Declare desired state; operators reconcile to it.

3.2 Database Operators (OLTP Sources)

Three OLTP database engines are required as data sources:

3.2.1 PostgreSQL – CloudNativePG Operator

CloudNativePG is the CNCF-endorsed production-grade PostgreSQL operator.

```yaml
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: pg-source-cluster
  namespace: platform-storage
spec:
  instances: 3                          # HA: 1 primary + 2 standbys
  primaryUpdateStrategy: unsupervised
  postgresql:
    parameters:
```

```
      wal_level: "logical"                # Enable CDC via logical replication
      max_replication_slots: "20"
  storage:
    size: 100Gi
    storageClass: gp3                     # AWS EBS gp3 volume
  bootstrap:
    initdb:
      database: source_db
      owner: nifi_user
  backup:
    retentionPolicy: "30d"
    barmanObjectStore:
      destinationPath: "s3://backups/cnpg/"
      s3Credentials:
        accessKeyId:
          name: aws-creds
          key: ACCESS_KEY_ID
```

Listing 3.1: CloudNativePG Cluster CRD – PostgreSQL

3.2.2 SQL Server – SQL Server Operator

Microsoft provides a Kubernetes operator for SQL Server 2022 via **mssql-operator**. Alternatively, a Helm-based deployment with persistent EBS volumes provides an MVP-grade solution.

⚠ MVP Note

For the MVP scope, SQL Server may be deployed as a single-instance StatefulSet backed by an EBS gp3 volume rather than a full HA operator, to reduce operational complexity within the project timeline.

3.2.3 IBM Informix

IBM Informix does not have a first-party Kubernetes Operator. The recommended approach for the MVP is:

- Deploy Informix as a **StatefulSet** using the official IBM Docker image (`ibmcom/informix-develop`
- Attach a persistent gp3 EBS volume for data storage.
- Expose via a `ClusterIP` service within the `platform-storage` namespace.

3.3 MinIO Operator – Object Storage

MinIO Operator is the official, production-grade operator for MinIO deployments on Kubernetes. It manages multi-tenant, high-availability MinIO clusters as first-class Kubernetes objects.

3.3.1 Installation via Helm

```
# Add MinIO Helm repository
helm repo add minio-operator https://operator.min.io
helm repo update

# Install the operator into its dedicated namespace
helm install minio-operator minio-operator/operator \
  --namespace minio-operator \
  --create-namespace \
  --set operator.replicaCount=2 \
  --version 6.0.0
```

Listing 3.2: MinIO Operator Helm Installation

3.3.2 MinIO Tenant CRD

```
apiVersion: minio.min.io/v2
kind: Tenant
metadata:
  name: lakehouse-tenant
  namespace: platform-storage
spec:
  image: "minio/minio:RELEASE.2024-01-18T22-51-28Z"
  pools:
    - name: pool-0
      servers: 4                          # 4 MinIO server pods
      volumesPerServer: 4                 # 4 EBS volumes per pod
          = 16 drives total
      volumeClaimTemplate:
        metadata:
          name: data
        spec:
          accessModes: ["ReadWriteOnce"]
          storageClassName: gp3
          resources:
```

```
            requests:
              storage: 500Gi              # 500Gi x 16 = ~8Ti
                 total raw capacity
  mountPath: /export
  requestAutoCert: true                   # TLS via cert-manager
  features:
    bucketDNS: true
    domains:
      minio:
        - minio.internal.example.com
  users:
    - name: minio-admin-secret
  buckets:
    - name: lakehouse
    - name: backups
    - name: terraform-state             # Separated from CF R2
       for internal use
```

Listing 3.3: MinIO Tenant CRD – Lakehouse Storage

Why EBS over S3 for MinIO?

Although MinIO can use AWS S3 as its backend storage (*gateway mode*), this project intentionally uses **EBS volumes**:

- **Performance**: EBS gp3 delivers consistent low-latency I/O; S3 Gateway introduces variable latency unsuitable for Iceberg commit operations.
- **Cloud Agnosticism**: The architecture is portable—replace EBS with Ceph RBD or vSAN to run on-premises without changing application configuration.
- **Cost Predictability**: EBS cost is deterministic; S3 request costs can spike with Iceberg's many small-file operations.

3.4 Apache Kafka Operator – Strimzi

Strimzi is the de-facto CNCF-graduated Kafka operator for Kubernetes.

```
apiVersion: kafka.strimzi.io/v1beta2
kind: Kafka
metadata:
  name: platform-kafka
  namespace: platform-ingestion
spec:
```

```yaml
  kafka:
    version: 3.7.0
    replicas: 3
    listeners:
      - name: plain
        port: 9092
        type: internal
        tls: false
      - name: tls
        port: 9093
        type: internal
        tls: true
    config:
      offsets.topic.replication.factor: 3
      transaction.state.log.replication.factor: 3
      log.retention.hours: 168
    storage:
      type: persistent-claim
      size: 200Gi
      class: gp3
  zookeeper:
    replicas: 3
    storage:
      type: persistent-claim
      size: 50Gi
      class: gp3
```

Listing 3.4: Strimzi Kafka Cluster CRD

3.5 Spark Operator

Kubeflow Spark Operator (formerly GoogleCloudPlatform/spark-on-k8s-operator) manages Spark jobs as `SparkApplication` CRDs.

```
helm repo add spark-operator \
  https://kubeflow.github.io/spark-operator

helm install spark-operator spark-operator/spark-operator \
  --namespace platform-compute \
  --create-namespace \
  --set webhook.enable=true \
```

```
  --set metrics.enable=true \
  --set metrics.prometheusPort=10254
```

Listing 3.5: Spark Operator Installation

```yaml
apiVersion: sparkoperator.k8s.io/v1beta2
kind: SparkApplication
metadata:
  name: bronze-iceberg-writer
  namespace: platform-compute
spec:
  type: Python
  pythonVersion: "3"
  mode: cluster
  image: "myregistry/spark-iceberg:3.5.0"
  imagePullPolicy: Always
  mainApplicationFile: "s3a://jobs/bronze_writer.py"
  sparkVersion: "3.5.0"
  sparkConf:
    "spark.sql.extensions": "org.apache.iceberg.spark.
       extensions.IcebergSparkSessionExtensions"
    "spark.sql.catalog.lakehouse": "org.apache.iceberg.spark.
       SparkCatalog"
    "spark.sql.catalog.lakehouse.type": "hive"
    "spark.sql.catalog.lakehouse.uri": "thrift://hive-
       metastore:9083"
    "spark.hadoop.fs.s3a.endpoint": "http://minio.platform-
       storage:9000"
    "spark.hadoop.fs.s3a.access.key": "${MINIO_ACCESS_KEY}"
    "spark.hadoop.fs.s3a.path.style.access": "true"
  driver:
    cores: 2
    coreLimit: "2000m"
    memory: "4g"
    serviceAccount: spark-sa
  executor:
    cores: 4
    instances: 3
    memory: "8g"
```

Listing 3.6: SparkApplication CRD – Iceberg Writer Job

3.6 Apache Airflow Operator

The official **Apache Airflow Helm Chart** (maintained by the Airflow community) is used for orchestration.

```
helm repo add apache-airflow https://airflow.apache.org
helm repo update

helm install airflow apache-airflow/airflow \
  --namespace platform-orchestr \
  --create-namespace \
  --set executor=KubernetesExecutor \
  --set dags.gitSync.enabled=true \
  --set dags.gitSync.repo="https://github.com/org/data-
     platform-dags" \
  --set dags.gitSync.branch=main \
  --set postgresql.enabled=false \
  --set data.metadataConnection.host=pg-airflow-rw.platform-
     storage
```

Listing 3.7: Airflow Installation via Helm

4 Ingestion Layer

4.1 Ingestion Strategy

The ingestion layer handles three distinct data patterns:

Table 4.1: Ingestion Pattern Matrix

Pattern	Source	Technology	Target
CDC (real-time)	PostgreSQL, SQL Server	NiFi + Debezium + Kafka + Spark SS	Bronze (Iceberg)
Batch	SQL queries, full table dumps	NiFi + Spark batch	Bronze (Iceberg)
File-based	CSV, XLSX, XLS	Python script + Spark	Bronze (Iceberg)

4.2 Apache NiFi – CDC and Batch Ingestion

Apache NiFi provides a visual, flow-based programming model for data routing and transformation. It is deployed as a StatefulSet with persistent storage for flow configuration and provenance.

NiFi Flow Design

CDC Flow (PostgreSQL / SQL Server):

1. `CaptureChangeMySQL` / `ListenCDC`: Subscribe to database logical replication slots.
2. `PublishKafka`: Publish raw CDC events to Kafka topic `cdc.{schema}.{table}`.
3. Kafka topic acts as durable buffer between NiFi and Spark.

Batch Flow:

1. `QueryDatabaseTable`: Execute incremental SQL queries based on watermark column.
2. `ConvertRecord`: Transform JDBC output to Avro or JSON.
3. `PublishKafka`: Publish to Kafka batch topic.

4.3 File-Based Ingestion (CSV / XLSX / XLS)

Flat files (CSV, XLSX, XLS) are uploaded to a designated MinIO bucket prefix (`s3a://landing/files/`) or to an AWS S3 staging bucket. A Python-based Spark job reads these files and writes them as Iceberg tables in the Bronze layer.

```yaml
apiVersion: sparkoperator.k8s.io/v1beta2
kind: SparkApplication
metadata:
  name: file-to-iceberg-ingestion
  namespace: platform-compute
spec:
  type: Python
  mainApplicationFile: "s3a://jobs/file_ingestion.py"
  arguments:
    - "--source-path=s3a://landing/files/"
    - "--target-table=lakehouse.bronze.raw_files"
    - "--file-format=xlsx"
  sparkConf:
    "spark.jars.packages": >
      org.apache.iceberg:iceberg-spark-runtime-3.5_2.12:1.4.2,
      com.crealytics:spark-excel_2.12:3.5.0_0.20.3
```

Listing 4.1: File Ingestion SparkApplication CRD

The Python ingestion script uses `spark-excel` for XLSX reading and writes directly to Iceberg:

```python
df = spark.read \
     .format("com.crealytics.spark.excel") \
     .option("header", "true") \
     .option("inferSchema", "true") \
     .load("s3a://landing/files/*.xlsx")

df.writeTo("lakehouse.bronze.raw_budget_files") \
  .tableProperty("write.format.default", "parquet") \
  .tableProperty("write.parquet.compression-codec", "zstd") \
  .createOrReplace()
```

Listing 4.2: Core file ingestion logic (pseudo-code)

4.4 Kafka – Streaming Buffer

Kafka serves as the durable, replayable message bus between NiFi (producers) and Spark Structured Streaming (consumers). Topic naming convention follows the pattern:

`<layer>.<source_system>.<schema>.<table>`

Example: `bronze.pg.public.agency_budget`

5 Storage Layer – Data Lakehouse

5.1 Apache Iceberg Table Format

Apache Iceberg is an open table format designed for analytic datasets at scale. It is the foundational storage format for all three Medallion layers.

Key capabilities leveraged in this project:

- **ACID transactions**: Concurrent reads and writes without data corruption.
- **Schema evolution**: Add, rename, drop columns without rewriting data.
- **Time travel**: Query historical snapshots of tables.
- **Partition evolution**: Change partitioning strategy without full rewrites.
- **Hidden partitioning**: Iceberg manages partitioning transparently, eliminating partition-specific query logic.

5.2 Hive Metastore

Apache Hive Metastore (HMS) serves as the central catalog for all Iceberg tables, storing schema, partition metadata, and table statistics. It is accessed by Spark, Dremio, and dbt via the Thrift protocol on port 9083.

```yaml
# charts/hive-metastore/values.yaml
image:
  repository: apache/hive
  tag: 4.0.0
database:
  host: pg-hms-rw.platform-metastore
  name: hive_metastore
  user: hive
  passwordSecret: hive-db-secret
warehouseDir: "s3a://lakehouse/warehouse/"
s3:
```

```
12  endpoint: "http://minio.platform-storage:9000"
13  accessKey:
14    secretName: minio-credentials
15    key: access-key
```

Listing 5.1: Hive Metastore Deployment (Helm values)

6 Compute Layer

6.1 dbt – Data Transformation

dbt (data build tool) manages Silver and Gold layer transformations as version-controlled SQL models. dbt runs on Spark via the `dbt-spark` adapter.

dbt Project Structure

```
dbt_project/
  models/
    silver/
      stg_agency_budget.sql       <- Cleaned budget data
      stg_clients.sql             <- Deduped client records
    gold/
      agg_monthly_budget.sql      <- Aggregated metrics
      dim_agency.sql              <- Agency dimension table
  macros/
    iceberg_helpers.sql
  dbt_project.yml
  profiles.yml
```

dbt jobs are triggered by **Airflow DAGs** as `SparkSubmitOperator` or `BashOperator` tasks, maintaining full lineage within the orchestration layer.

6.2 JupyterHub – Interactive Analytics

JupyterHub provides a multi-user Jupyter environment for data scientists and analysts. It is deployed via the official Zero to JupyterHub Helm chart, with notebooks running as ephemeral Kubernetes pods (`KubeSpawner`).

Users connect to MinIO/Iceberg data directly from notebooks using PySpark sessions, enabling exploratory data analysis on Silver/Gold data without impacting production Spark jobs.

7 Exposure & Security – Cloudflare Zero Trust

7.1 Architecture Rationale

Exposing Kubernetes services to external users traditionally requires configuring cloud load balancers, public IP addresses, TLS certificates, and firewall rules. **Cloudflare Tunnel** (formerly Argo Tunnel) provides a more secure, cost-effective alternative:

- **No inbound firewall rules**: The `cloudflared` daemon initiates an outbound tunnel to Cloudflare's edge. No ports need to be opened on the EKS cluster.
- **Zero Trust access policies**: Every service is protected by Cloudflare Access (identity-aware proxy).
- **Free tier**: Cloudflare Tunnel with Zero Trust is available at no cost for the scale of an MVP project.
- **Built-in DDoS protection** and TLS termination at Cloudflare's global edge.

7.2 Cloudflare Operator Installation

```
helm repo add cloudflare \
  https://cloudflare.github.io/helm-charts
helm repo update

helm install cloudflare-operator cloudflare/cloudflare-
   operator \
  --namespace platform-security \
  --create-namespace \
  --set config.apiToken.secretName=cloudflare-api-token \
  --set config.accountId="YOUR_CF_ACCOUNT_ID"
```

Listing 7.1: Cloudflare Operator Installation

7.3 TunnelBinding CRD

The `TunnelBinding` CRD binds a DNS hostname (managed by Cloudflare) to a Kubernetes `Service`:

```yaml
apiVersion: networking.cfargotunnel.com/v1alpha1
kind: TunnelBinding
metadata:
  name: airflow-tunnel
  namespace: platform-orchestr
spec:
  subjects:
    - name: airflow-webserver
      spec:
        fqdn: airflow.dataplatform.example.com
        protocol: http
        servicePort: 8080
        noTLSVerify: false
  tunnelRef:
    kind: ClusterTunnel
    name: platform-cloudflare-tunnel
```

Listing 7.2: TunnelBinding CRD – Airflow Webserver

Zero Trust Application Policy

Each exposed service (Airflow, JupyterHub, Dremio, Grafana) is registered as a **Cloudflare Access Application** with the following policy:

- **Identity Provider**: GitHub SSO (organization membership check).
- **Service Auth**: Machine-to-machine authentication via Cloudflare Service Tokens.
- **Posture checks**: Device certificate validation for production services.

8 Virtualization & BI – Dremio and Power BI

8.1 Dremio Nautilus

Dremio is a data lakehouse query engine that provides:

- **Apache Arrow Flight**: High-throughput columnar data transport between Dremio and BI tools.
- **Virtual Datasets**: Create SQL views over Iceberg tables without data movement.
- **Reflections**: Automatic query acceleration via materialized aggregations.
- **RBAC**: Fine-grained row and column-level security policies.

Dremio is deployed via the official **Dremio Helm chart**, with coordinator and executor pods separated for independent scaling.

8.1.1 Dremio to Hive Metastore Connection

```
# Dremio REST API source payload
{
  "name": "lakehouse",
  "type": "HIVE",
  "config": {
    "hostname": "hive-metastore.platform-metastore",
    "port": 9083,
    "enableAsync": true,
    "propertyList": [
      {"name": "fs.s3a.endpoint",
       "value": "http://minio.platform-storage:9000"},
      {"name": "fs.s3a.path.style.access",
       "value": "true"},
      {"name": "fs.s3a.impl",
```

```
        "value": "org.apache.hadoop.fs.s3a.S3AFileSystem"}
    ]
  }
}
```

Listing 8.1: Dremio Source Configuration – Hive Metastore

8.2 Power BI Integration

Power BI connects to Dremio via the **ODBC/JDBC connector** or the native **Apache Arrow Flight** connector:

1. Install the **Dremio ODBC driver** on the Power BI Gateway machine.
2. Configure the data source to point to `dremio.dataplatform.example.com:31010`.
3. Authenticate using Dremio PAT (Personal Access Token).
4. Import Gold-layer datasets and publish reports to Power BI Service.

9 Observability Layer

9.1 Three Pillars of Observability

Table 9.1: Observability Pillars

Pillar	Purpose	Tool
Metrics	Numeric time-series for latency, throughput, saturation	Prometheus + Grafana
Logs	Structured event records from all Kubernetes workloads	Fluent Bit + OpenObserve
Traces	Distributed request tracing across services	OpenTelemetry (future)

9.2 Prometheus & Grafana

Prometheus is deployed via the `kube-prometheus-stack` Helm chart, which bundles Prometheus, AlertManager, node exporter, and a curated set of Grafana dashboards.

```
helm repo add prometheus-community \
  https://prometheus-community.github.io/helm-charts

helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace platform-monitoring \
  --create-namespace \
  --set prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=false \
  --set alertmanager.alertmanagerSpec.storage.volumeClaimTemplate.spec.storageClassName=gp3 \
  --set prometheus.prometheusSpec.storageSpec.volumeClaimTemplate.spec.storageClassName=gp3 \
  --set prometheus.prometheusSpec.retention=30d
```

Listing 9.1: kube-prometheus-stack Installation

ServiceMonitor CRDs are created for each platform component to configure Prometheus scrape targets automatically:

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: spark-operator-monitor
  namespace: platform-monitoring
  labels:
    release: monitoring
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: spark-operator
  namespaceSelector:
    matchNames: ["platform-compute"]
  endpoints:
    - port: metrics
      interval: 30s
      path: /metrics
```

Listing 9.2: ServiceMonitor for Spark Operator

9.3 OpenObserve – Log Management

OpenObserve is a cloud-native, open-source log analytics platform that serves as the Elasticsearch/Kibana replacement.

9.3.1 Log Pipeline Architecture

Kubernetes Pod Logs → Fluent Bit → S3 (AWS) → OpenObserve

Fluent Bit is deployed as a `DaemonSet` on every EKS node. It tails container logs, enriches them with Kubernetes metadata (pod name, namespace, labels), and ships them to S3 in compressed JSON format.

OpenObserve is configured with an **S3 input source**, continuously ingesting the log files deposited by Fluent Bit. This decoupled, S3-buffered architecture ensures log durability even if OpenObserve is unavailable.

```
[OUTPUT]
    Name                          s3
    Match                         kube.*
```

```
    bucket                          platform-logs
    region                          eu-west-1
    s3_key_format                   /year=%Y/month=%m/day=%d/%H-%
       M-%S.gz
    total_file_size                 50M
    upload_timeout                  10m
    compression                     gzip
    json_date_key                   _timestamp
    json_date_format                iso8601
```

Listing 9.3: Fluent Bit ConfigMap (excerpt)

10 | Infrastructure as Code – Terraform & Ansible

10.1 Terraform – Cloud Infrastructure

Terraform provisions and manages all cloud infrastructure components:

- **AWS EKS Cluster**: Control plane, managed node groups, OIDC provider.
- **EBS Volumes**: gp3 storage classes and associated IAM policies.
- **IAM Roles**: IRSA (IAM Roles for Service Accounts) for S3, EBS, Lambda.
- **VPC**: Subnets, security groups, NAT gateway.
- **Lambda Functions**: Node group scaling automation.
- **S3 Buckets**: Fluent Bit log destination, Terraform state (local to AWS).

10.1.1 Terraform State on Cloudflare R2

Terraform remote state is stored on **Cloudflare R2** to reduce dependency on AWS. R2 is S3-compatible and free up to 10 GB/month.

```
terraform {
  backend "s3" {
    bucket                      = "terraform-state-pfe"
    key                         = "dataplatform/terraform.
       tfstate"
    region                      = "auto"
    endpoint                    = "https://<ACCOUNT_ID>.r2.
       cloudflarestorage.com"
    skip_credentials_validation = true
    skip_metadata_api_check     = true
    skip_region_validation      = true
    force_path_style            = true
```

```
  }
}
```

Listing 10.1: Terraform Backend Configuration – Cloudflare R2

10.1.2 Terraform Module Structure

Repository Layout

```
terraform/
  modules/
    eks/                # EKS cluster, node groups, addons
    vpc/                # VPC, subnets, security groups
    iam/                # IRSA roles, policies
    lambda-scaling/     # Node group scheduling Lambda
    s3/                 # Log bucket, landing bucket
  environments/
    dev/
      main.tf
      variables.tf
      terraform.tfvars
    prod/
      main.tf
```

10.2 Ansible – Configuration Management

Ansible handles application-level configuration that falls outside Terraform's scope:

- Installing and configuring the `kubectl`, `helm`, and `aws-cli` on CI runners.
- Post-Terraform bootstrap: initializing Kubernetes namespaces and applying RBAC manifests.
- Helm chart values file generation (templated with Ansible Jinja2).
- Secret seeding into AWS Secrets Manager (referenced by External Secrets Operator in K8s).

11 | CI/CD Pipeline – GitHub Actions

11.1 Pipeline Overview

GitHub Actions Workflow Stages

1. **Lint & Validate**: `terraform fmt`, `terraform validate`, `tflint`.
2. **Plan** (on PR): `terraform plan` output posted as PR comment.
3. **Apply** (on merge to `main`): `terraform apply -auto-approve`.
4. **Helm Deploy**: Apply updated Helm charts to EKS via `helm upgrade -install`.
5. **Smoke Test**: Basic connectivity tests to ensure services are healthy post-deploy.

```yaml
name: Terraform CI/CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  terraform:
    runs-on: ubuntu-latest
    permissions:
      id-token: write          # For OIDC authentication to AWS
      contents: read
      pull-requests: write

    steps:
      - uses: actions/checkout@v4

      - name: Configure AWS Credentials (OIDC)
```

```
      uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: arn:aws:iam::ACCOUNT:role/github-
            actions-role
        aws-region: eu-west-1

    - name: Setup Terraform
      uses: hashicorp/setup-terraform@v3
      with:
        terraform_version: 1.7.0

    - name: Terraform Init
      run: terraform init
      working-directory: terraform/environments/dev
      env:
        AWS_ACCESS_KEY_ID:     ${{ secrets.CF_R2_ACCESS_KEY
            }}
        AWS_SECRET_ACCESS_KEY: ${{ secrets.CF_R2_SECRET_KEY
            }}

    - name: Terraform Plan
      run: terraform plan -out=tfplan

    - name: Terraform Apply
      if: github.ref == 'refs/heads/main'
      run: terraform apply tfplan
```

Listing 11.1: GitHub Actions – Terraform Plan/Apply Workflow

12 | Cost Optimization – EKS Node Scheduling

12.1 Lambda-Based Node Group Scheduling

EKS worker nodes incur EC2 costs even when idle. To minimize cost during the MVP phase, **AWS Lambda functions** trigger EKS Managed Node Group scaling on a schedule:

Table 12.1: Node Scaling Schedule

Event	Time (UTC+1)	Action	Trigger
Scale Up	08:00 AM	Set min/desired nodes to N	EventBridge cron
Scale Down	06:00 PM	Set min/desired nodes to 0	EventBridge cron

```
import boto3

def handler(event, context):
    eks = boto3.client('eks', region_name='eu-west-1')
    ec2 = boto3.client('autoscaling', region_name='eu-west-1')

    # Get the ASG associated with the EKS node group
    response = eks.describe_nodegroup(
        clusterName='data-platform-cluster',
        nodegroupName='compute-nodegroup'
    )
    asg_name = response['nodegroup']['resources']['
       autoScalingGroups'][0]['name']

    # Scale up
    ec2.update_auto_scaling_group(
        AutoScalingGroupName=asg_name,
        MinSize=2,
```

```
        DesiredCapacity=3,
        MaxSize=10
    )
    return {"status": "scaled_up", "nodegroup": "compute-
        nodegroup"}
```

Listing 12.1: Lambda Scale-Up Function (Python excerpt)

Important Considerations

- The Kubernetes **control plane** (EKS managed) remains running 24/7 ($0.10/hr). Only worker node EC2 instances are scaled to zero.
- **Stateful workloads** (MinIO, PostgreSQL) must be on separate node groups with `nodeSelector` to avoid disruption during scale-down.
- Consider setting MinIO node group `MinSize=1` to preserve data availability outside working hours.

13 | Optional – ArgoCD GitOps

13.1 GitOps with ArgoCD

OPTIONAL **ArgoCD** provides GitOps-based continuous delivery for Kubernetes manifests and Helm charts. If implemented, it replaces the `helm upgrade` step in GitHub Actions with a declarative pull-based model.

ArgoCD Application Pattern

Each platform component is defined as an `Application` CRD pointing to a Git repository path. ArgoCD continuously reconciles the cluster state to match the Git-declared desired state. Drift is auto-detected and can be auto-remediated.

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: minio-operator
  namespace: argocd
spec:
  project: data-platform
  source:
    repoURL: https://github.com/org/data-platform-gitops
    targetRevision: HEAD
    path: helm/minio-operator
    helm:
      valueFiles:
        - values/dev.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: minio-operator
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

```
22    syncOptions:
23      - CreateNamespace=true
```

Listing 13.1: ArgoCD Application CRD – MinIO Operator

14 Implementation Roadmap

14.1 MVP Phased Delivery Plan

Table 14.1: Project Milestones

Phase	Duration	Deliverables	Priority
1	Week 1–2	VPC, EKS cluster via Terraform. Namespace & RBAC setup. GitHub Actions CI. CF R2 backend.	REQUIRED
2	Week 3–4	MinIO Operator + Tenant. Hive Metastore. CloudNativePG (PostgreSQL). Cloudflare Tunnel.	REQUIRED
3	Week 5–6	Strimzi Kafka. NiFi deployment. First end-to-end CDC pipeline to Bronze Iceberg table.	REQUIRED
4	Week 7–8	Spark Operator. dbt Silver transforms. Airflow orchestration. File ingestion script.	REQUIRED
5	Week 9–10	Dremio deployment & HMS connection. Power BI dataset import. Gold layer models.	REQUIRED
6	Week 11	Prometheus + Grafana. Fluent Bit + OpenObserve. Lambda scaling. SQL Server/Informix.	MVP
7	Week 12	JupyterHub. ArgoCD. Performance tuning. Documentation. Demo preparation.	OPTIONAL

14.2 Risk Register

Table 14.2: Project Risk Register

Risk	Descri	Likelihood	Mitigation
EKS Cost Overrun	Node groups running beyond schedule	Medium	Lambda scaling, billing alerts
Iceberg Conflicts	Concurr write failures under load	Low	Optimistic concurrency in Spark config
NiFi Complexity	Flow design longer than expected	Medium	Start with batch; add CDC later
Informix Operator	No native K8s operator available	High	Use StatefulSet + EBS as workaround
Timeline Overrun	12-week scope too ambitious	Medium	Phase 7 is optional; cut if needed

15 Best Practices Summary

Engineering Best Practices Applied

Immutable Infrastructure All infrastructure is defined in code (Terraform). No manual changes to production.

Least Privilege IAM IRSA assigns per-service-account AWS IAM roles with minimal required permissions.

Secret Management Secrets are never stored in Git. AWS Secrets Manager + External Secrets Operator injects them at runtime.

Network Isolation Each namespace has its own `NetworkPolicy`. Cross-namespace traffic is explicitly allowed, not implicit.

Backup & Recovery CloudNativePG continuous WAL archiving to S3. MinIO Operator supports scheduled backups.

Observability-First Every operator deployment includes a `ServiceMonitor` and structured log format from day one.

GitOps Ready All Helm values and manifests are version-controlled. Every change is a PR, reviewed, and auditable.

Cloud Agnosticism MinIO (not S3), Iceberg (not Delta Lake proprietary), K8s (not ECS/Fargate) keep the stack portable.

References

1. Apache Iceberg Documentation. https://iceberg.apache.org/docs/
2. MinIO Kubernetes Operator. https://min.io/docs/minio/kubernetes/upstream/
3. Strimzi Kafka Operator. https://strimzi.io/documentation/
4. CloudNativePG Operator. https://cloudnative-pg.io/documentation/
5. Kubeflow Spark Operator. https://www.kubeflow.org/docs/components/spark-operator/
6. Dremio Documentation. https://docs.dremio.com/
7. Cloudflare Tunnel & Zero Trust. https://developers.cloudflare.com/cloudflare-one/
8. kube-prometheus-stack. https://github.com/prometheus-community/helm-charts
9. OpenObserve Documentation. https://openobserve.ai/docs/
10. Terraform Cloudflare R2 Backend. https://developers.cloudflare.com/r2/
11. Apache Airflow Helm Chart. https://airflow.apache.org/docs/helm-chart/
12. AWS EKS Best Practices Guide. https://aws.github.io/aws-eks-best-practices/